

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) pre_proc... - pre_pr.csv

pre_process_datasample - pre_process_datasample.xlsx - pre_process_datasample - pre_pr.csv(text/csv) - 226 bytes, last modified: 9/16/2026 - 100% done

Saving pre_process_datasample - pre_process_datasample.xlsx - pre_process_datasample - pre_pr.csv to pre_process_datasample

```
import numpy as np
import pandas as pd
```

```
df = pd.read_csv('/content/pre_process_datasample - pre_process_datasample.xlsx - pre_process_datasample - pre_pr.csv')
df
```

	Country	Age	Salary	Purchased	
0	France	44.0	72000.0	No	
1	Spain	27.0	48000.0	Yes	
2	Germany	30.0	54000.0	No	
3	Spain	38.0	61000.0	No	
4	Germany	40.0	NaN	Yes	
5	France	35.0	58000.0	Yes	
6	Spain	NaN	52000.0	No	
7	France	48.0	79000.0	Yes	
8	Germany	50.0	83000.0	No	
9	France	37.0	67000.0	Yes	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Country     10 non-null    object
1   Age         9 non-null     float64
2   Salary      9 non-null     float64
3   Purchased   10 non-null    object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
df.Country.mode()
```

```
Country
0   France

dtype: object
```

```
df.Country.mode()[0]
```

```
'France'
```

```
type(df.Country.mode())
```

```
pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None,
fastpath: bool | lib.NoDefault=lib.no_default) -> None
```

Note that the index is first build with the keys from the dictionary.
After this the Series is reindexed with the given Index values, hence we
get all NaN as a result.

Constructing Series from a list with `copy=False`.

```
>>> r = [1, 2]
```

```
df.Country.fillna(df.Country.mode()[0], inplace=True)
df.Age.fillna(df.Age.median(), inplace=True)
```

```
df.Salary.fillna(round(df.Salary.mean()), inplace=True)
```

```
df
```

p/ipykernel_664/3065201449.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
f.Country.fillna(df.Country.mode()[0], inplace=True)
```

p/ipykernel_664/3065201449.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
f.Age.fillna(df.Age.median(), inplace=True)
```

p/ipykernel_664/3065201449.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
f.Salary.fillna(round(df.Salary.mean()), inplace=True)
```

Country	Age	Salary	Purchased	
France	44.0	72000.0	No	
Spain	27.0	48000.0	Yes	
Germany	30.0	54000.0	No	
Spain	38.0	61000.0	No	
Germany	40.0	63778.0	Yes	
France	35.0	58000.0	Yes	
Spain	38.0	52000.0	No	
France	48.0	79000.0	Yes	
Germany	50.0	83000.0	No	
France	37.0	67000.0	Yes	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
pd.get_dummies(df.Country)
```

	France	Germany	Spain	
0	True	False	False	
1	False	False	True	
2	False	True	False	
3	False	False	True	
4	False	True	False	
5	True	False	False	
6	False	False	True	
7	True	False	False	
8	False	True	False	
9	True	False	False	

```
updated_dataset = pd.concat(  
    [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],  
    axis=1  
)
```

```
updated_dataset
```

	France	Germany	Spain	Age	Salary	Purchased	
0	True	False	False	44.0	72000.0	No	
1	False	False	True	27.0	48000.0	Yes	
2	False	True	False	30.0	54000.0	No	
3	False	False	True	38.0	61000.0	No	
4	False	True	False	40.0	63778.0	Yes	
5	True	False	False	35.0	58000.0	Yes	
6	False	False	True	38.0	52000.0	No	
7	True	False	False	48.0	79000.0	Yes	
8	False	True	False	50.0	83000.0	No	
9	True	False	False	37.0	67000.0	Yes	

Next steps: [Generate code with updated_dataset](#)[New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   Country     10 non-null     object
 1   Age         10 non-null     float64
 2   Salary      10 non-null     float64
 3   Purchased   10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
updated_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
```

updated_dataset

/tmp/ipykernel_664/3592612786.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c[...] The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
updated_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
/tmp/ipykernel_664/3592612786.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
updated_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
```

	France	Germany	Spain	Age	Salary	Purchased	
0	True	False	False	44.0	72000.0	0	
1	False	False	True	27.0	48000.0	1	
2	False	True	False	30.0	54000.0	0	
3	False	False	True	38.0	61000.0	0	
4	False	True	False	40.0	63778.0	1	
5	True	False	False	35.0	58000.0	1	
6	False	False	True	38.0	52000.0	0	
7	True	False	False	48.0	79000.0	1	
8	False	True	False	50.0	83000.0	0	
9	True	False	False	37.0	67000.0	1	

Next steps: [Generate code with updated_dataset](#)[New interactive sheet](#)

